Kütüphane Yönetim Sistemi (v1): Mimari, Diyagramlar ve Yol Haritası Kılavuzu

Teknoloji: C# / ASP.NET Core / Entity Framework Core / SQL Server

Bu kılavuz; kapsamı, gereksinimleri, mimari deseni, üç UML diyagramını (Class, Object, Deployment), veritabanı tasarımını ve uygulama yol haritasını tek yerde toplar. Kapsam bilinçli olarak **v1** ile sınırlandırılmıştır; şube, rezervasyon, bağış, envanter sayımı, üyelik türleri ve üye self-servisi gibi özellikler v2 listesinde tutulur.

1. Kapsam

1.1 v1 kapsamındakiler

ID	Modül	İçerik
FR-1	Ana panel	Toplam kitap sayısı, toplam üye sayısı, ödünçteki kitap sayısı, gecikmiş kitap sayısı, son ödünç işlemleri
FR-2	Kitap yönetimi	Ekleme, düzenleme, silme (pasife alma); ad, yazar, ISBN, yayınevi, yayın yılı, kategori, açıklama; toplam kopya sayısı; kopya barkodu; listeleme, arama; müsait / ödünçte durumu
FR-3	Üye yönetimi	Ekleme, düzenleme, silme (pasife alma); ad soyad, e-posta, telefon, üye numarası, kayıt tarihi; aktif / pasif; üye arama; üyenin ödünç aldığı kitapları görüntüleme
FR-4	Ödünç ve iade	Üye seçme, kitap seçme, ödünç tarihi, otomatik teslim tarihi, iade alma, iade tarihini kaydetme, süresi geçmişleri belirleme, aynı kopyanın tekrar verilmesini engelleme
FR-5	Arama ve filtreleme	Kitap adı, yazar, kategori, üye adı; ödünçteki kitaplar; gecikmiş kitaplar
FR-6	Basit raporlama	En çok ödünç alınan kitaplar, en çok kitap alan üyeler, ödünçteki kitaplar, gecikmiş kitaplar, tarih aralığına göre ödünç işlemleri, CSV / Excel çıktısı (isteğe bağlı)

1.2 Aktörler

- **Yönetici (Admin):** Tüm işlemler, personel hesapları, ayarlar (ödünç süresi vb.)
- **Kütüphaneci (Librarian):** Kitap, üye, ödünç ve iade işlemleri, raporlar

v1'de üye sisteme giriş yapmaz; tüm işlemleri personel yürütür.

1.3 Fonksiyonel olmayan gereksinimler

- **Güvenlik:** Parolalar hash'li saklanır (ASP.NET Core Identity), rol bazlı yetkilendirme, CSRF ve XSS korumaları
- **Veri bütünlüğü:** Ödünç işlemleri transaction içinde yapılır; veritabanında benzersiz kısıtlar bulunur
- **Performans:** Liste ekranlarında sayfalama (paging); arama alanlarında indeks
- **Kullanılabilirlik:** Barkod okuyucu klavye gibi davrandığı için barkod alanı odaklı hızlı işlem akışı
- **Bakım:** Katmanlı yapı, bağımlılık enjeksiyonu, birim testler

1.4 v2 listesi (şimdilik dışarıda)

Şube yönetimi, rezervasyon, ceza yönetimi, üyelik türleri ve limitleri, üye self-servis hesabı, bağış ve satın alma süreçleri, envanter sayımı, toplu içe aktarma, bildirimler (e-posta / SMS), tatil günü politikası, ayıklama süreci, yedekleme izleme.

2. Mimari

2.1 Seçilen desen

Katmanlı (N-Tier) mimari + Repository / Unit of Work + Service katmanı, ASP.NET Core MVC üzerinde.

Katman	Sorumluluk	Örnek içerik
Presentation	Arayüz, istek yönetimi	Controller'lar, Razor View'lar, ViewModel'ler
Business	İş kuralları, doğrulama	`BookService`, `MemberService`, `LoanService`, `ReportService`, `DashboardService`, DTO'lar
Domain	Çekirdek modeller	Entity'ler, enum'lar, repository arayüzleri
Data Access	Veritabanı erişimi	`LibraryDbContext`, repository implementasyonları, migration'lar

Bağımlılık yönü: Presentation → Business → Domain ← Data Access. Domain katmanı hiçbir katmana bağımlı değildir. Business katmanı repository'leri arayüz üzerinden kullanır; somut sınıflar Dependency Injection ile enjekte edilir.

2.2 Çözüm (Solution) yapısı

```
Library.sln
├── Library.Domain          (Entities, Enums, Interfaces)
├── Library.Data            (DbContext, Repositories, Migrations)
├── Library.Business        (Services, DTOs, Validators)
├── Library.Web             (Controllers, Views, ViewModels, wwwroot)
└── Library.Tests           (xUnit, Moq)
```

2.3 Teknoloji yığını

Alan	Tercih
Dil / çatı	C#, .NET (güncel LTS), ASP.NET Core MVC
ORM	Entity Framework Core (Code First)
Veritabanı	SQL Server
Kimlik	ASP.NET Core Identity (roller: Admin, Librarian)
Doğrulama	FluentValidation (veya Data Annotations)
Loglama	Serilog
Test	xUnit + Moq
CSV / Excel	CsvHelper / ClosedXML
Sürüm kontrolü	Git

2.4 Kullanılan tasarım ilkeleri

- **SOLID**, özellikle tek sorumluluk (SRP) ve bağımlılığın tersine çevrilmesi (DIP)
- **DTO / ViewModel:** Entity'ler doğrudan arayüze taşınmaz
- **Soft delete:** Kayıtlar fiziksel silinmez, pasife alınır (`IsDeleted` / `IsActive`)
- **Hesaplanan değerler saklanmaz:** Müsait kopya sayısı ve gecikme durumu sorguyla / özellikle hesaplanır

3. İş kuralları

3.1 Temel ilke

Ödünç verilen şey kitabın kendisi değil, **fiziksel bir kopyasıdır**. Bu yüzden `Book` (eser) ile `BookCopy` (fiziksel nüsha) ayrıdır ve `Loan` kaydı `BookCopyId`'ye bağlanır.

3.2 Ödünç verme (`LoanService.BorrowBook`)

1. Üyenin aktif olduğunu kontrol et
2. Seçilen kopyanın `Available` olduğunu kontrol et
3. Transaction başlat
4. Aktif ödünç kaydı (`Loan`) oluştur
5. Teslim tarihini hesapla: `DueDate = LoanDate + LoanPeriodDays` (gün sayısı ayardan okunur)
6. Kopya durumunu `Loaned` yap
7. Kaydet ve transaction'ı tamamla
8. İşlem sonucunu kullanıcıya göster

Her ihlal için açık bir sonuç kodu dönülür: `MemberInactive`, `CopyNotAvailable`, `CopyNotFound`.

3.3 İade (`LoanService.ReturnBook`)

1. Kopyanın aktif ödünç kaydını bul (`ReturnDate == null`)
2. İade tarihini kaydet
3. Kopya durumunu `Available` yap
4. Transaction'ı tamamla
5. İşlem sonucunu göster (gecikme varsa gün sayısı ile birlikte)

3.4 Hesaplanan durumlar

- **Müsait kopya sayısı:** `Status == Available` olan `BookCopy` sayısı (saklı sayaç yoktur)
- **Toplam kopya sayısı:** Kitaba bağlı, silinmemiş `BookCopy` sayısı
- **Gecikmiş ödünç:** `ReturnDate == null && DueDate < bugün`
- **Ödünçteki kitap sayısı:** `ReturnDate == null` olan `Loan` sayısı

3.5 Çift ödünç engeli (iki katman)

- **Servis katmanı:** Kontrol ve yazma tek transaction içinde yapılır

- **Veritabanı:** `Loan` tablosunda `BookCopyId` üzerinde filtrelenmiş benzersiz indeks (`WHERE ReturnDate IS NULL`). İki personel aynı anda aynı kopyayı vermeye çalışsa bile ikincisi veritabanı tarafından reddedilir

3.6 Silme kuralları

- Ödünç geçmişi olan kitap, kopya veya üye fiziksel silinmez; pasife alınır
- Aktif ödüncü olan üye pasife alınamaz (veya uyarı verilir); ödünçteki kopya silinemez

4. Class diyagramı

4.1 Nasıl okunur

- `+` public, `-` private
- `*--` kompozisyon: `BookCopy`, `Book` olmadan var olamaz
- `-->` ilişki (association); uçlardaki sayılar çokluktur (1, 0.., *1..*)
- `Book` ile `Author` çoktan çoğadır; veritabanında `BookAuthor` ara tablosu oluşur

4.2 Diyagram (Mermaid kaynağı)

```
classDiagram
    class Book {
        +int Id
        +string Title
        +string ISBN
        +int PublishYear
        +string Description
        +int CategoryId
        +int PublisherId
        +bool IsDeleted
        +GetAvailableCopyCount() int
    }
    class BookCopy {
        +int Id
        +string Barcode
        +CopyStatus Status
        +int BookId
        +bool IsDeleted
    }
    class Author {
        +int Id
        +string FullName
    }
```

```
class Publisher {
    +int Id
    +string Name
}
class Category {
    +int Id
    +string Name
}
class Member {
    +int Id
    +string MemberNumber
    +string FullName
    +string Email
    +string Phone
    +DateTime RegisteredAt
    +bool IsActive
    +CanBorrow() bool
}
class Loan {
    +int Id
    +DateTime LoanDate
    +DateTime DueDate
    +DateTime? ReturnDate
    +int MemberId
    +int BookCopyId
    +int StaffId
    +IsOverdue() bool
}
class Staff {
    +int Id
    +string FullName
    +string Email
    +Role Role
}
class CopyStatus {
    <<enumeration>>
    Available
    Loaned
    Lost
    Damaged
}
class Role {
    <<enumeration>>
    Admin
    Librarian
}
```

```
    Book "1" *-- "1..*" BookCopy
    Book "*" -- "*" Author
    Book "*" --> "1" Publisher
    Book "*" --> "1" Category
    BookCopy "1" --> "0..*" Loan
    Member "1" --> "0..*" Loan
    Staff "1" --> "0..*" Loan
    BookCopy ..> CopyStatus
    Staff ..> Role
```

4.3 Servis ve repository yapısı (ikinci Class diyagramı)

İş mantığı sınıfları, entity diyagramından ayrı bir diyagramda gösterilir; böylece her diyagram tek bir derdi anlatır.

```
classDiagram
    class ILoanService {
        <<interface>>
        +BorrowBook(memberId, copyId, staffId) LoanResult
        +ReturnBook(copyId) ReturnResult
        +GetOverdueLoans() List~Loan~
        +GetActiveLoans() List~Loan~
    }
    class LoanService {
        -ILoanRepository loans
        -IBookCopyRepository copies
        -IMemberRepository members
        -IUnitOfWork uow
    }
    class ILoanRepository {
        <<interface>>
        +GetActiveByCopy(copyId) Loan
        +GetOverdue() List~Loan~
        +Add(loan)
    }
    class LoanRepository
    class LoansController {
        -ILoanService service
        +Borrow() IActionResult
        +Return() IActionResult
    }
```

```
    ILoanService <|.. LoanService
    ILoanRepository <|.. LoanRepository
    LoanService ..> ILoanRepository
    LoansController ..> ILoanService
```

Aynı kalıp `BookService`, `MemberService`, `ReportService` ve `DashboardService` için tekrarlanır.

5. Object diyagramı

5.1 Nasıl okunur

Object diyagramı, Class diyagramının **belirli bir andaki örnek halidir**. Kutu başlığı `nesneAdı : SınıfAdı` biçimindedir; alanlar gerçek değerleriyle yazılır; çokluk yazılmaz; metot gösterilmez. Her nesne ve bağlantının Class diyagramında karşılığı olan bir sınıf ve ilişkisi bulunmalıdır.

5.2 Senaryo

Tarih: **17 Ekim 2026**. Kütüphaneci Zeynep, üye Ali'ye iki kopya vermiştir. Birinin teslim tarihi geçmiştir, diğeri süresi içindedir. "Kürk Mantolu Madonna" kitabının ikinci kopyası rafta müsaittir.

5.3 Nesneler

Nesne	Alan değerleri
`zeynep : Staff`	FullName = "Zeynep Kaya", Role = Librarian
`ali : Member`	MemberNumber = "M-0001", FullName = "Ali Yılmaz", IsActive = true
`kitap1 : Book`	Title = "Kürk Mantolu Madonna", PublishYear = 1943
`kitap2 : Book`	Title = "Suç ve Ceza", PublishYear = 1866
`yazar1 : Author`	FullName = "Sabahattin Ali"
`yazar2 : Author`	FullName = "Fyodor Dostoyevski"
`kopya1 : BookCopy`	Barcode = "B-0001", Status = Loaned
`kopya2 : BookCopy`	Barcode = "B-0002", Status = Available
`kopya3 : BookCopy`	Barcode = "B-0101", Status = Loaned

Nesne	Alan değerleri
`odunc1 : Loan`	LoanDate = 01.10.2026, DueDate = 15.10.2026, ReturnDate = null (**gecikmiş**)
`odunc2 : Loan`	LoanDate = 10.10.2026, DueDate = 24.10.2026, ReturnDate = null (süresinde)

5.4 Bağlantılar

`kitap1` — `kopya1`, `kopya2` · `kitap2` — `kopya3` · `kitap1` — `yazar1` · `kitap2` — `yazar2` · `odunc1` — `ali`, `kopya1`, `zeynep` · `odunc2` — `ali`, `kopya3`, `zeynep`

5.5 Diyagram (Mermaid kaynağı)

```
classDiagram
    class zeynep["zeynep : Staff"] {
        FullName = Zeynep Kaya
        Role = Librarian
    }
    class ali["ali : Member"] {
        MemberNumber = M-0001
        FullName = Ali Yilmaz
        IsActive = true
    }
    class kitap1["kitap1 : Book"] {
        Title = Kurk Mantolu Madonna
        PublishYear = 1943
    }
    class kitap2["kitap2 : Book"] {
        Title = Suc ve Ceza
        PublishYear = 1866
    }
    class kopya1["kopya1 : BookCopy"] {
        Barcode = B-0001
        Status = Loaned
    }
    class kopya2["kopya2 : BookCopy"] {
        Barcode = B-0002
        Status = Available
    }
    class kopya3["kopya3 : BookCopy"] {
        Barcode = B-0101
        Status = Loaned
    }
```

```
    class odunc1["odunc1 : Loan"] {
        LoanDate = 2026-10-01
        DueDate = 2026-10-15
        ReturnDate = null
    }
    class odunc2["odunc2 : Loan"] {
        LoanDate = 2026-10-10
        DueDate = 2026-10-24
        ReturnDate = null
    }
    kitap1 -- kopya1
    kitap1 -- kopya2
    kitap2 -- kopya3
    odunc1 -- ali
    odunc1 -- kopya1
    odunc1 -- zeynep
    odunc2 -- ali
    odunc2 -- kopya3
    odunc2 -- zeynep
```

Kontrol: `odunc1` için `ReturnDate == null` ve `DueDate < 17.10.2026` olduğundan `IsOverdue()` true döner; ana paneldeki "gecikmiş kitap sayısı" 1'dir. `kitap1` için toplam kopya 2, müsait kopya 1'dir.

6. Deployment diyagramı

6.1 Nasıl okunur

Deployment diyagramı yazılımın **çalışma ortamındaki fiziksel dağılımını** gösterir: hangi artifact (dosya / derleme) hangi düğümde (node) çalışır ve düğümler hangi protokolle haberleşir. Düğümler üç boyutlu kutu ile çizilir; bağlantıların üzerine protokol yazılır.

6.2 Düğümler ve artifact'lar

Düğüm	Tür	İçerik (artifact)
İstemci bilgisayarı	«device»	Web tarayıcı; (isteğe bağlı) barkod okuyucu
Web sunucusu	«device»	«execution environment» Kestrel veya IIS: `Library.Web.dll`, `Library.Business.dll`, `Library.Domain.dll`, `Library.Data.dll`, `appsettings.json`

Düğüm	Tür	İçerik (artifact)
Veritabanı sunucusu	«device»	«execution environment» SQL Server: `LibraryDB` veritabanı
Yedekleme deposu (isteğe bağlı)	«device»	Gece yedekleri (`.bak`)

6.3 Bağlantılar

- İstemci → Web sunucusu: **HTTPS (443)**
- Web sunucusu → Veritabanı sunucusu: **TCP/IP (1433), EF Core / TDS**, şifreli bağlantı
- Veritabanı sunucusu → Yedekleme deposu: **dosya aktarımı** (zamanlanmış)

6.4 Diyagram (Mermaid kaynağı)

```
flowchart LR
    subgraph C["«device» İstemci Bilgisayarı"]
        B["Web Tarayıcı"]
        S["Barkod Okuyucu (isteğe bağlı)"]
    end
    subgraph W["«device» Web Sunucusu"]
        subgraph K["«execution environment» Kestrel / IIS"]
            A1["Library.Web.dll"]
            A2["Library.Business.dll"]
            A3["Library.Domain.dll"]
            A4["Library.Data.dll"]
            A5["appsettings.json"]
        end
    end
    subgraph D["«device» Veritabanı Sunucusu"]
        subgraph Q["«execution environment» SQL Server"]
            DB[("LibraryDB")]
        end
    end
    subgraph Y["«device» Yedekleme Deposu (isteğe bağlı)"]
        BK["gece yedekleri .bak"]
    end
    S --> B
    B -- "HTTPS 443" --> A1
    A4 -- "TCP/IP 1433 (EF Core / TDS)" --> DB
    DB -. "zamanlanmış yedek" .-> BK
```

Geliştirme aşamasında üç düğüm tek bilgisayarda (localhost) çalışabilir; diyagram yine de hedef (canlı) yerleşimi göstermelidir.

7. Veritabanı tasarımı

7.1 Tablolar

Tablo	Önemli sütunlar	Notlar
`Books`	Id, Title, ISBN, PublishYear, Description, CategoryId, PublisherId, IsDeleted	ISBN için indeks
`BookCopies`	Id, Barcode, Status, BookId, IsDeleted	`Barcode` benzersiz
`Authors`	Id, FullName	
`BookAuthors`	BookId, AuthorId	Bileşik birincil anahtar
`Publishers`	Id, Name	
`Categories`	Id, Name	
`Members`	Id, MemberNumber, FullName, Email, Phone, RegisteredAt, IsActive	`MemberNumber` benzersiz
`Loans`	Id, LoanDate, DueDate, ReturnDate, MemberId, BookCopyId, StaffId	Filtrelenmiş benzersiz indeks
`Staff` / Identity tabloları	Kullanıcı, rol, parola hash'i	ASP.NET Core Identity
`Settings`	Key, Value	Örn. `LoanPeriodDays = 14`

7.2 Kritik kısıtlar

```
-- Bir kopya aynı anda yalnızca tek aktif ödünçte olabilir
CREATE UNIQUE INDEX UX_Loans_ActiveCopy
    ON Loans (BookCopyId)
    WHERE ReturnDate IS NULL;
```

- `BookCopies.Barcode`: benzersiz
- `Members.MemberNumber`: benzersiz
- Yabancı anahtarlarda silme davranışı `Restrict` (zincirleme silme yok)
- Arama sütunlarında (Title, FullName) indeks

7.3 Kopya durum geçişleri (v1)

Mevcut durum	Geçerli sonraki durumlar
Available	Loaned, Lost, Damaged
Loaned	Available (iade), Lost
Damaged	Available (onarım sonrası), Lost
Lost	Available (bulunursa)

Geçişler serviste kontrol edilir; durum alanı serbest metin gibi kullanılmaz.

8. Ekranlar (arayüz envanteri)

Ekran	İçerik
Giriş	Personel girişi
Ana panel	FR-1 sayıları, son ödünç işlemleri, gecikmişlere kısayol
Kitap listesi / arama	Ad, yazar, kategori filtresi; müsait / ödünçte rozeti; sayfalama
Kitap ekle / düzenle	Bilgiler, yazar seçimi, kopya ekleme (barkod)
Üye listesi / arama	Ad arama, aktif / pasif filtresi
Üye ekle / düzenle	Bilgiler, durum
Üye detay	Mevcut ve geçmiş ödünçler
Ödünç ver	Üye seç, barkod okut, teslim tarihi önizleme, onay
İade al	Barkod okut, gecikme bilgisi, onay
Gecikmiş kitaplar	Liste, gecikme gün sayısı
Raporlar	FR-6 raporları, tarih aralığı, CSV dışa aktarma
Ayarlar (Admin)	Ödünç süresi, personel hesapları

9. Yol haritası

Süreler haftada yaklaşık 10-15 saat çalışan tek geliştirici için tahminidir; kendi tempoya göre ölçekleyin. Her fazın sonunda "bitti" ölçütü vardır; ölçüt sağlanmadan sonraki faza geçilmez.

Faz 0: Hazırlık (3-4 gün)

- Visual Studio / VS Code, .NET SDK, SQL Server (LocalDB veya Express), Git kurulumu
- Git deposu oluşturma, `main` ve özellik dalları (feature branch) stratejisi
- Bu kılavuzun depoda `docs/` klasörüne konulması

Bitti ölçütü: Boş çözüm derleniyor, depoya itildi.

Faz 1: Analiz ve diyagramların kesinleştirilmesi (1 hafta)

- Bu kılavuzdaki gereksinimlerin gözden geçirilmesi, eksik / belirsiz maddelerin netleştirilmesi
- Class, Object ve Deployment diyagramlarının draw.io veya Mermaid ile temiz çizimi, PNG / SVG dışa aktarımı
- Ekran taslakları (kâğıt veya Figma / draw.io wireframe)
- ER diyagramının Class diyagramından türetilmesi

Bitti ölçütü: Diyagramlar birbiriyle tutarlı (Object diyagramındaki her nesne Class diyagramında karşılık buluyor).

Faz 2: Çözüm iskeleti ve veri katmanı (1 hafta)

- Beş projenin oluşturulması ve proje referanslarının mimariye göre bağlanması
- Domain: entity'ler, enum'lar, repository arayüzleri
- Data: `LibraryDbContext`, Fluent API yapılandırmaları, ilişkiler, benzersiz ve filtrelenmiş indeksler
- İlk migration, veritabanının oluşturulması
- Seed data: kategoriler, birkaç yazar / yayınevi / kitap, kopyalar, bir yönetici hesabı
- Repository implementasyonları ve Unit of Work

Bitti ölçütü: Veritabanı migration ile sıfırdan kurulabiliyor; repository'ler temel CRUD işlemlerini yapıyor.

Faz 3: İş mantığı katmanı (1,5-2 hafta)

- `BookService`, `MemberService` (CRUD, arama, soft delete)

- `LoanService.BorrowBook` ve `ReturnBook` (transaction, sonuç kodları, ayardan okunan ödünç süresi)
- `DashboardService`, `ReportService` sorguları
- DTO'lar ve doğrulama kuralları
- **Birim testler:** özellikle ödünç / iade senaryoları (pasif üye, müsait olmayan kopya, çift ödünç, gecikmeli iade, olmayan kopya)

Bitti ölçütü: İş kuralları arayüz olmadan testlerle doğrulanmış durumda.

Faz 4: Kimlik doğrulama ve arayüz iskeleti (1 hafta)

- ASP.NET Core Identity, roller (Admin, Librarian), giriş / çıkış
- Ortak layout, menü, hata ve bildirim (toast) yapısı
- Yetki kontrolleri (`[Authorize(Roles = ...)]`)

Bitti ölçütü: Giriş yapmadan hiçbir sayfaya erişilemiyor; rol kısıtları çalışıyor.

Faz 5: Ekranların geliştirilmesi (2 hafta)

Sıra: kitap yönetimi → üye yönetimi → ödünç ver / iade al → arama ve filtreler → ana panel → raporlar.

- Sayfalama, sıralama ve boş durum mesajları
- Ödünç ve iade ekranlarında barkod odaklı hızlı akış
- Her ekranda sunucu tarafı doğrulama ve anlaşılır hata mesajları

Bitti ölçütü: FR-1'den FR-5'e kadar tüm maddeler arayüzden uçtan uca çalışıyor.

Faz 6: Raporlama ve dışa aktarma (3-4 gün)

- FR-6 raporları, tarih aralığı filtresi
- CSV (isteğe bağlı Excel) dışa aktarma

Bitti ölçütü: Her rapor, veritabanındaki gerçek veriyle elle kontrol edilen sonuç veriyor.

Faz 7: Test, güvenlik ve cilalama (1 hafta)

- Entegrasyon testleri (test veritabanı ile)
- Eşzamanlılık testi: aynı kopyaya iki eş zamanlı ödünç denemesi, yalnızca birinin başarılı olması
- Güvenlik gözden geçirmesi: yetkilendirme, CSRF, girdi doğrulama, parola politikası, hassas veri loglanmaması
- Loglama (Serilog) ve global hata yakalama

- Performans: büyük veriyle (örn. 10.000 kitap) liste ve arama hızı

Bitti ölçütü: Test listesi geçiyor; bilinen kritik hata yok.

Faz 8: Yayına alma ve dokümantasyon (3-4 gün)

- Deployment diyagramındaki yerleşime göre kurulum (IIS veya Docker)
- Üretim `appsettings` ve gizli bilgilerin (bağlantı dizesi) güvenli yönetimi
- Veritabanı yedekleme planı
- README, kurulum kılavuzu, kullanıcı kılavuzu, final diyagram seti

Bitti ölçütü: Sistem, geliştirme makinesi dışında bir ortamda baştan kurulup çalıştırılabiliyor.

Toplam

Yaklaşık **8-10 hafta**. Zaman sıkışırsa Faz 6'daki Excel çıktısı ve Faz 7'deki performans testi esnetilebilir; Faz 3 ve Faz 7'deki ödünç / iade ve eşzamanlılık testleri esnetilmemelidir.

10. Test planı (özet)

Senaryo	Beklenen sonuç
Aktif üyeye müsait kopya ödünç ver	Loan oluşur, kopya `Loaned` olur, teslim tarihi = bugün + ödünç süresi
Pasif üyeye ödünç ver	`MemberInactive`, kayıt oluşmaz
Ödünçteki kopyayı tekrar ver	`CopyNotAvailable`, kayıt oluşmaz
Aynı kopyaya eş zamanlı iki ödünç	Yalnızca biri başarılı; veritabanı kısıtı ikincisini reddeder
Zamanında iade	ReturnDate yazılır, kopya `Available` olur
Gecikmeli iade	İade başarılı; gecikme gün sayısı gösterilir
Ödünçte olmayan kopyayı iade al	Anlaşılır hata, veri değişmez
Ödünç geçmişi olan kitabı sil	Fiziksel silinmez, pasife alınır
Aktif ödüncü olan üyeyi pasife al	Uyarı / engel
Gecikmiş listesi	Yalnızca `ReturnDate == null && DueDate < bugün` kayıtları

11. Diyagram çizim kontrol listesi

Class diyagramı

- Her sınıfın adı isim (tekil, PascalCase), özellikleri ve gerekli metotları yazılı
- İlişki türleri doğru: kompozisyon, ilişki, kalıtım, bağımlılık, gerçekleme
- Her ilişkinin iki ucunda çokluk var
- Getter / setter ve yardımcı metotlar gösterilmemiş
- Entity diyagramı ile servis diyagramı ayrılmış

Object diyagramı

- Başlıklar `nesne : Sınıf` biçiminde, alt çizgili
- Alan değerleri gerçekçi ve tutarlı (gecikmiş ödünç gerçekten gecikmiş)
- Çokluk ve metot yok
- Her bağlantının Class diyagramında karşılığı var

Deployment diyagramı

- Düğümlerin türü (device / execution environment) belirtilmiş
- Artifact'lar doğru düğümün içinde
- Her bağlantıda protokol ve port var
- Canlı ortam gösteriliyor; yedekleme gibi isteğe bağlı parçalar açıkça işaretli

Araçlar: draw.io (diagrams.net) UML şablonlarıyla, Mermaid / PlantUML (kod olarak sürümlenebilir), Visual Studio Class Designer (koddan Class diyagramı üretmek için).

12. v2 yol haritası (öneri sırası)

1. Ceza yönetimi ve üyelik türüne göre ödünç limitleri
2. Rezervasyon
3. Üye self-servis hesabı ve e-posta bildirimleri
4. Birden çok şube
5. Bağış, satın alma talebi ve ayıklama süreçleri
6. Envanter sayımı, toplu içe / dışa aktarma
7. Tatil günü politikası ile teslim tarihi hesabı

Bu özelliklerin çoğu mevcut modeli bozmadan eklenebilir: ceza `Loan`'a bağlı yeni bir tablo olur, rezervasyon `Book` ve `Member`'a bağlanır, şube `BookCopy`'ye `BranchId` ekler.